

# Real-Time Stock Market Data Analysis Using Kafka

Step-by-Step Architecture, Setup, Ingestion, Crawling, and SQL Querying Pipeline

## 1. Architecture Overview & Prerequisites

This guide details the end-to-end execution steps for building a **Real-Time Stock Market Data Processing Engine** using **Apache Kafka** on **AWS EC2**, streaming into **Amazon S3**, cataloging via **AWS Glue**, and analyzing via **Amazon Athena**. The pipeline simulates real-time stock ticker events, ingests streaming payloads, and enables serverless SQL queries.

| Phase            | Component / Service              | Role in Pipeline                                                                      |
|------------------|----------------------------------|---------------------------------------------------------------------------------------|
| 1. Cluster Setup | AWS EC2 & Apache Kafka           | Hosts Zookeeper & Kafka Broker to manage distributed event topics.                    |
| 2. Producer      | Python (Jupyter / KafkaProducer) | Simulates stock ticker events from CSV and pushes JSON stream to Kafka.               |
| 3. Consumer      | Python (KafkaConsumer & s3fs)    | Consumes streamed messages from Kafka topic and writes JSON files to S3.              |
| 4. Cataloging    | AWS S3 & AWS Glue Crawler        | Stores raw streaming JSON objects; Glue infers dynamic schema & updates Data Catalog. |
| 5. Analytics     | Amazon Athena (SQL)              | Executes ad-hoc SQL queries on streaming stock data cataloged in Glue.                |

**Prerequisites:** AWS Account with Administrator/IAM privileges, Python 3.5+ (Jupyter Notebook), and packages: pandas, kafka-python, s3fs, and awscli.

## 2. Major Step 1: AWS EC2 Provisioning & Kafka Server Setup

### 1.1 Launch AWS EC2 Instance & Connect:

- Launch a **t2.micro** instance (Amazon Linux 2, Free Tier) on AWS EC2 Console.
- Create key pair (e.g., kafka-key.pem) and allow Security Group inbound rules: **SSH (Port 22)** and **Custom TCP (Port 9092)**.
- Connect via terminal: `chmod 400 kafka-key.pem && ssh -i kafka-key.pem ec2-user@<EC2_PUBLIC_IP>`.

### 1.2 Install Kafka & Java Runtime:

```
wget https://archive.apache.org/dist/kafka/2.8.0/kafka_2.12-2.8.0.tgz
tar -xzf kafka_2.12-2.8.0.tgz && cd kafka_2.12-2.8.0
sudo yum install java-1.8.0-openjdk -y
```

### 1.3 Configure Advertised Listeners & Start Cluster:

- Edit `config/server.properties`: set `advertised.listeners=PLAINTEXT://<EC2_PUBLIC_IP>:9092`.
- **Window 1 (Zookeeper)**: `bin/zookeeper-server-start.sh config/zookeeper.properties`
- **Window 2 (Kafka Broker)**: `export KAFKA_HEAP_OPTS="-Xmx512M -Xms256M" && bin/kafka-server-start.sh config/server.properties`
- **Window 3 (Topic Creation)**: `bin/kafka-topics.sh --create --topic demo_test --bootstrap-server <EC2_PUBLIC_IP>:9092 --replication-factor 1 --partitions 1`

## 3. Major Step 2: Streaming Data Simulation & Kafka Producer

---

Simulates real-time stock ticker events by sampling historical records (indexProcessed.csv), converting to JSON objects, and publishing to the Kafka topic.

**2.1 Dependencies:** pip install kafka-python pandas s3fs

**2.2 Producer Logic (KafkaProducer.ipynb):**

```
import pandas as pd
from kafka import KafkaProducer
from json import dumps
import time

producer = KafkaProducer(
    bootstrap_servers=['<EC2_PUBLIC_IP>:9092'],
    value_serializer=lambda x: dumps(x).encode('utf-8')
)
df = pd.read_csv('indexProcessed.csv')
while True:
    dict_stock = df.sample(1).to_dict(orient="records")[0]
    producer.send('demo_test', value=dict_stock)
    time.sleep(1) # 1-second simulated interval
```

## 4. Major Step 3: Kafka Consumer & Ingestion into AWS S3

---

**3.1 Setup AWS IAM Credentials & S3 Bucket:**

- Create an S3 Bucket (e.g., kafka-stock-market-tutorial-unique).
- Configure IAM credentials locally using aws configure.

**3.2 Consumer Logic (KafkaConsumer.ipynb):**

Reads messages from topic demo\_test and writes JSON files directly to Amazon S3 using s3fs:

```
from kafka import KafkaConsumer
from json import loads, dumps
from s3fs import S3FileSystem

consumer = KafkaConsumer(
    'demo_test',
    bootstrap_servers=['<EC2_PUBLIC_IP>:9092'],
    value_deserializer=lambda x: loads(x.decode('utf-8'))
)
s3 = S3FileSystem()
for count, i in enumerate(consumer):
    with s3.open(f"s3://kafka-stock-market-tutorial-unique/stock_market_{count}.json", 'w') as file:
        dumps(i.value, file)
```

## 5. Major Step 4: AWS Glue Schema Crawling & Data Cataloging

---

- **Create Glue IAM Role:** Attach `AWSGlueServiceRole` and `AmazonS3FullAccess`.
- **Configure Glue Crawler:** Target S3 path `s3://kafka-stock-market-tutorial-unique/`, output database `stock_market_kafka`.
- **Run Crawler:** Scans JSON objects, extracts schema (Index, Date, Open, High, Low, Close, Volume), and creates catalog table.

## 6. Major Step 5: Real-Time Analytics with Amazon Athena

---

- **Configure Query Location:** Set S3 output directory under Athena Settings.
- **Execute SQL Queries:** Query live stream records as JSON files arrive continuously in S3:

```
-- Check live stream ingestion record count
SELECT COUNT(*) FROM "stock_market_kafka"."kafka_stock_market_tutorial_unique";

-- Query latest 10 stock ticker records
SELECT index, date, open, high, low, close, volume
FROM "stock_market_kafka"."kafka_stock_market_tutorial_unique"
ORDER BY date DESC LIMIT 10;
```

### SUMMARY OF END-TO-END PIPELINE:

1. **Producer Stream:** Python Producer reads CSV rows → serializes to JSON → pushes to Kafka topic on EC2.
2. **Kafka Broker:** Distributed queue managing topic partitions and real-time streaming payloads.
3. **Consumer Ingestion:** Python Consumer reads stream → auto-saves individual event JSON files into AWS S3.
4. **Glue & Athena:** Glue maintains schema catalog; Athena executes instant serverless SQL on incoming S3 JSON files.